

Runbook — Notes API Operations & Incident Response

Purpose: Practical, copy-paste remediation steps for common incidents. Written to be usable under pressure (the "3 AM on-call" test). **Scope:** single-node k3s cluster hosting the Notes API + PostgreSQL.

0. Access & Prerequisites

```
# SSH into the VM
ssh -i ~/Downloads/sre-test-key.pem ubuntu@13.126.63.217

# All kubectl/helm commands on the VM use this prefix:
#   sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl ...
```

Key endpoints

- App: <http://13.126.63.217>
 - Grafana: <http://13.126.63.217:30080> (admin / admin123)
 - Prometheus / Alerts: <http://13.126.63.217:30090/alerts>
-

1. First 5 Minutes of ANY Incident (triage)

Do these first, in order, before touching anything:

```
# 1. What's the overall state? Anything NOT Running/Completed shows up here:
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods -A | grep -Ev
"Running|Completed"

# 2. App pods specifically
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods -l app=notes-api

# 3. Is the app answering?
curl -s -o /dev/null -w "HTTP %{http_code}\n" http://13.126.63.217/health

# 4. Which alerts are firing? (open in browser)
#   http://13.126.63.217:30090/alerts

# 5. Recent errors in the logs
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl logs -l app=notes-api --tail=50
| grep -i error
```

Decision guide:

- [/health](#) returns 200 but [/api/notes](#) fails → **database problem** → Section 3

- Pods in **CrashLoopBackOff** → **bad config / bad image** → Section 2
- High CPU alert firing → **resource exhaustion** → Section 4
- Everything down / node unreachable → **node/cluster problem** → Section 6

2. Pod Crash Loop / CrashLoopBackOff

Symptoms: pods restarting repeatedly; **NotesApiPodCrashLooping** firing; possibly **NotesApiDown** and **NotesApiPodNotReady** too.

Diagnose

```
# See restart counts and status
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods -l app=notes-api

# Why is it crashing? Check the last logs of the crashing pod:
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl logs <POD_NAME> --previous --tail=50

# Describe the pod for events (bad image, OOM, failed probe, etc.):
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl describe pod <POD_NAME>
```

Remediate

```
# If a recent deploy caused it → roll back to the previous good release:
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml helm history notes-app
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml helm rollback notes-app <PREVIOUS_REVISION>

# If it's a transient issue → restart the deployment:
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl rollout restart deployment notes-api
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl rollout status deployment notes-api
```

Verify

```
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods -l app=notes-api #
all 1/1 Running
curl -s http://13.126.63.217/health
```

Note: with 2 replicas, a bad config crashes the *new* replica set while the old healthy pods keep serving (HA). Prioritise rollback over panic — users may not be impacted yet.

3. Database Connectivity Loss

Symptoms: `/api/notes` returns HTTP 503 "database unavailable"; `/health` still 200;
`NotesApiHighErrorRate` firing; logs show `connection refused to notes-api-postgres:5432`.

Diagnose

```
# Is the Postgres pod running?
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods -l app=notes-api-postgres

# Postgres logs
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl logs -l app=notes-api-postgres -tail=50

# Confirm the app's error (connection refused vs timeout tells you a lot):
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl logs -l app=notes-api --tail=30 | grep -i "database\|connection"
```

- **"connection refused"** → Postgres process/pod is down → restore it (below).
- **"timed out"** → network/NetworkPolicy/DNS issue → check Service + policies.

Remediate

```
# If Postgres is scaled down or crashed → scale it back up:
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl scale deployment notes-api-postgres --replicas=1
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl rollout status deployment notes-api-postgres

# If the Postgres pod is stuck → delete it (it will be recreated with the PVC intact):
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl delete pod -l app=notes-api-postgres
```

Verify

```
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods -l app=notes-api-postgres # 1/1 Running
curl -s http://13.126.63.217/api/notes | head -c 200
# returns notes JSON
```

Note: the app recovers automatically once the DB is back — no app restart needed. Data is safe on the `PersistentVolumeClaim`.

4. High CPU / Resource Exhaustion

Symptoms: NotesApiHighCPU firing; elevated latency; pods still Running.

Diagnose

```
# Which pods are hot?
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl top pods -l app=notes-api

# Node-level pressure
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl top nodes

# What traffic is driving it? (check logs / Grafana request-rate panel)
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl logs -l app=notes-api --tail=50
| grep -i stress
```

Remediate

```
# Scale out to spread the load across more replicas:
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl scale deployment notes-api --
replicas=4

# If a specific abusive endpoint is the cause (e.g. /api/stress) → block/rate-
limit
# it at the ingress, or (temporarily) restart to drop in-flight work:
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl rollout restart deployment
notes-api

# Scale back once load subsides:
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl scale deployment notes-api --
replicas=2
```

Note: pod CPU *limits* (1 core each) prevent a single pod from starving the node. CPU pressure throttles requests; it does not OOM-kill (that's memory). The alert may take ~3 min to auto-resolve after load stops (metric averaging window).

5. Common Operations (day-to-day)

```
# Restart the app (rolling, zero-downtime with 2+ replicas)
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl rollout restart deployment
notes-api

# Scale up / down
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl scale deployment notes-api --
replicas=3

# Roll back a bad release
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml helm history notes-app
```

```
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml helm rollback notes-app <REVISION>

# Redeploy latest (normally CI/CD does this on git push to main)
cd ~/SRE-Project1 && git pull origin main
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml helm upgrade --install notes-app
./helm/notes-app --wait

# Tail live logs
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl logs -f -l app=notes-api

# Delete a stuck pod (safely recreated)
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl delete pod <POD_NAME>
```

6. Node / Cluster Down (single-node caveat)

Symptoms: nothing reachable; SSH may still work but kubectl fails.

```
# Is k3s running?
sudo systemctl status k3s

# Restart k3s if it's down
sudo systemctl restart k3s

# Check node status once it's back
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get nodes
```

Note: this is a **single-node** deployment — the node is a single point of failure. There is no failover to another node. If the VM itself is down, recovery means starting the EC2 instance (AWS Console) or, in the worst case, re-provisioning and letting CI/CD redeploy. In production this tier would run across multiple nodes.

7. Health Check (run anytime, e.g. before a demo)

```
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get nodes
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods -A | grep -Ev
"Running|Completed"
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods -n monitoring
curl -s -o /dev/null -w "app /health: HTTP %{http_code}\n"
http://13.126.63.217/health
# Then eyeball: app UI, Grafana dashboard, Prometheus alerts (all should be green)
```

8. Escalation & References

- **RCA**s for past incidents: [docs/rca-01-pod-crashloop.md](#), [docs/rca-02-database-outage.md](#), [docs/rca-03-high-cpu.md](#)
- **Design decisions & known limitations:** [docs/design-notes.md](#)
- **Security notes:** [docs/security.md](#)
- **Alert definitions:** [helm/notes-app/templates/prometheusrule.yaml](#)
- **Known limitation:** database and node are single points of failure in this PoC (documented; production would use HA Postgres + multi-node cluster).